CS 303 - Amortized Analysis

Amortization :- Payment of an obligation in a series of installments or transfers.

→ Guarantees the average performance of each operation in the worst case.

→ Calculate the upper bound $T(n)$ of n operations

The average cost = $\frac{T(n)}{n}$ } → amortized cost.

Three common techniques!

1. Aggregate analysis
2. Accounting method
3. Potential method.

Example : Stack with multipop.

Push (S, x) → pushes object x onto stack S.

Pop (S) → Pops the top of stack S, returns the popped object.

Multipop (S, k) → removes the k top objects (if they are available.) for stack S.

k is positive.

```
Multipop (S, k)
    While not Stack_Empty (S) and k > 0
        Pop (S).
        k = k - 1 ;
```

Assuming Push and Pop takes O(1) times.

So Multipop(S, k) takes O(k) times.

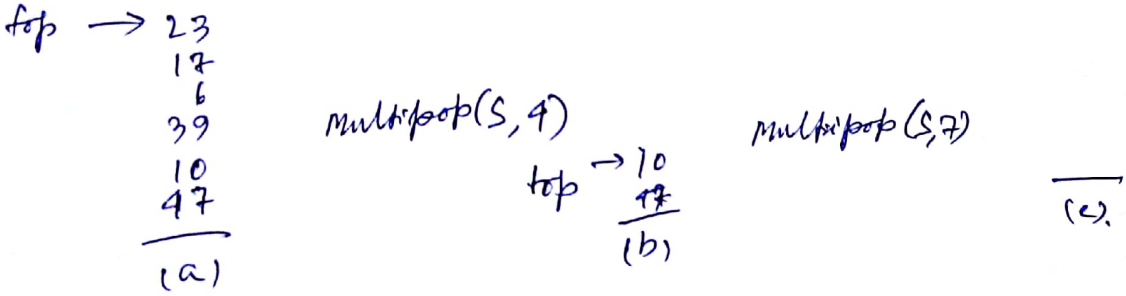
If we have a sequence

For a sequence of n Push, Pop, and Multipop operations on an empty stack.

The worst case time complexity of Multipop is O(n).

⇒ Worst case time complexity of any operation is O(n)

⇒ For n operations it is $O(n^2)$.



Aggregate analysis:

Number of Pop() operations (including those inside Multipop) = Number of Push() operations

From n operations on empty stack ⇒ At most n Push().

⇒ Number of Pop() ≤ n

Total time taken by n operations = O(n)

The average cost for each operation = O(n)/n = O(1).

CS 503 - Amortized Analysis ..

Accounting Method:

For each operation o_i, we pay an amortized cost $\rightarrow \hat{c}_i$

Actual cost $\rightarrow c_i$

$\hat{c}_i - c_i$ is credit for that operation. $\Rightarrow$ We can use that credit for future operations

$\rightarrow$ We have to produce an upper bound of total cost.

So,

$$\sum_{i=1}^{n} \hat{c}_i - \sum_{i=n}^{n} c_i$$ must be non-negative for any n.

MultiPop Example

Op	Actual cost	Amortized cost
Push	1	2
Pop	1	0
Multipop (S, k)	min (k, s) ↗ size of the stack	0

$\rightarrow$ Every Push save 1 credit. [Prepaid for future costly ops]

$\rightarrow$ For a next Pop we can use that 1 credit.

$\rightarrow$ For any Pop or Multipop we always save sufficient credit.

So total Amortized cost = $O(n)$.

$\Rightarrow$ Amortized cost per operation = $O(1)$.

The potential method :

Assign a potential function to the whole Data structure.

After each operation potential may change.

Potential function $\Phi(D_i) \rightarrow \mathbb{R}$

(D_i : after i^{th} op.)

Amortized cost $\hat{c}_i$ of the i^{th} op $=$

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

(c_i : Actual cost.)

So, $$\sum_{i=1}^{n} \hat{c}_i = \sum_{i=1}^{n} \left(c_i + \Phi(D_i) - \Phi(D_{i-1}) \right)$$

$$= \sum_{i=1}^{n} c_i + \Phi(D_n) - \Phi(D_0).$$

If we can define potential function such that $\Phi(D_n) \geq \Phi(D_0)$.

then, total amortized cost is an upper bound on the total cost.

→ sometimes we prove $\Phi(D_i) \geq \Phi(D_0)$ for any i.

Example :- Multipop stack.

Potential of a stack = Number of element in stack.

So, $\Phi(D_0) = 0$

and $\Phi(D_i) \geq \Phi(D_0)$ always = the min size of stack is 0.

After a Push : Potential difference is

$$\phi(D_i) - \phi(D_{i-1}) = (S+1) - S = 1.$$

⇒ Amortized cost of this Push op is

$$\hat{c}_i = c_i + \phi(D_i) - \phi(D_{i-1})$$
$$= 1 + 1$$
$$= 2$$

After a Multipop (S, k). [let $k' = \min(k, s)$], Potential difference is

$$\phi(D_i) - \phi(D_{i-1}) = -k'$$

~~Thus, the amor~~

Thus, $\hat{c}_i = c_i + \phi(D_i) - \phi(D_{i-1})$

$$= k' - k'$$
$$= 0$$

Similarly for Pop, $\hat{c}_i = 0$.

⇒ Amortized cost for each operation is $O(1)$.

⇒ Total amortized cost $= O(n)$

Already seen that $\phi(D_i) \geq \phi(D_0)$ for any i.

So, $\sum_{i=1}^{n} \hat{c}_i \geq \sum_{i=1}^{n} c_i$

Average cost ~~for~~ : $O(1)$.